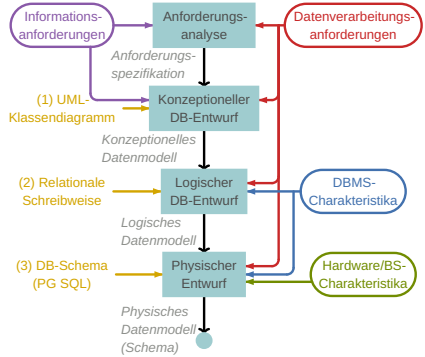
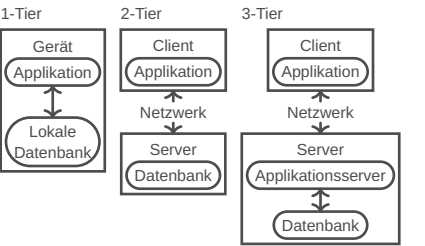




GLOSSAR	
Begriff	Bedeutung
Impedance-Mismatch	Diskrepanz zwischen Datenstrukturen auf Applikations- und Datenbankenebene
System-/Datenkatalog	Enthält Metadaten über die Datenbankobjekte, z.B. Tabellen und Schemata.
Datenbankschema	Struktur einer Datenbank, die die Organisation der Daten und Beziehungen beschreibt.
Datenbasis	Der physische Speicherort
Surrogate Key	Künstlich generierter PK
Referentielle Integrität	Fremdschlüssel muss zu einem Wert der referenzierten Tabelle oder NULL zeigen
Datenunabhängigkeit	Daten in einer DB ändern können, ohne dass Anwendungen geändert werden müssen
Data Pages	Kleinste Speicher-Dateneinheiten einer DB
Heaps	Unsortierte Datenorganisation
Semantische Integrität	Daten sind nicht nur syntaktisch, sondern auch inhaltlich korrekt, insbesondere nach T
Data dictionary	Zentrale Sammlung von Metadaten über die Daten im DBMS

DATENBANKMODELLE	
Begriff	Bedeutung
Hierarchisch	Daten sind in einer baumartigen Struktur geordnet
Netzwerk	Flexiblere Struktur als hierarchisch, Erlaubt mehrere Pfade zwischen Entitäten
Objektorientiert	Speichert Daten und ihr Verhalten in Form von Obj.
Objektrelational	Kombiniert objektorientierte + relationale Prinzipien
Relational	Speichert Daten in Tabellen (Relationen) und verwaltet Beziehungen durch Schlüssel



DATABASE SYSTEM (DBS)	
Besteht aus DBMS und Datenbasen	
DATABASE MANAGEMENT SYSTEM (DBMS)	
– (A) Transaktionen	– Datentypen
– (C) Konsistenz	– Abfragesprache
– (I) Mehrbenutzerbetrieb	– Backup & Recovery
– (D) Grosse Datenmengen	– Redundanzfreiheit
– (D) Sicherheit	– Kapselung

ANSI MODELL	
Logische Ebene: Logische Struktur der Daten	
Interne Ebene: Speicherstrukturen, Definition durch internes Schema (Beziehungen, Tabellen etc.)	
Externe Ebene: Sicht einer Benutzerklasse auf Teilmenge der DB, Definition durch externes Schema	
Mapping: Zwischen den Ebenen ist eine mehr oder weniger komplexe Abbildung notwendig	
RELATIONALES MODELL	
PK sind <u>unterstrichen</u> , FK sind <i>kursiv</i>	
tabellenname (	
id SERIAL PRIMARY KEY,	
grade DECIMAL(2,1) NOT NULL,	
fk INT FOREIGN KEY REFERENCES t2,	
u VARCHAR(9) DEFAULT CURRENT_USER,	
);	
UNIFIED MODELING LANGUAGE (UML)	
Eins	Mehrere
Eins (zwingend)	Mehrere (mind. 1)
0 oder eins	0 oder mehrere
Assoziation	Komposition
Aggregation	Vererbung

Complete: Alle Subklassen sind definiert
Incomplete: Zusätzliche Subklassen sind erlaubt
Disjoint: Ist Instanz von genau einer Unterklasse
Overlapping: Kann Instanz von mehreren überlappenden Unterklassen sein

NORMALISIERUNG
1NF: Atomare Attributwerte: *track* aufteilen

id	track
1	Fugazi: Song #1

⇒

id	interpret	titel
1	Fugazi	Song #1

2NF: Nichtschlüsselattr. voll vom Schlüssel abhängig.
Ist PK atomar, dann 2NF gegeben. Im Beispiel sind nicht alle Attribute des PK notwendig, um *album* eindeutig zu identifizieren

track	cd_id	album	titel
1	1	Repeater	Turnover
2	1	Repeater	Song #1

⇒ track

track	cd_id	titel
1	1	Turnover
2	1	Song #1

cd

id	album
1	Repeater

3NF: Keine transitiven Abhängigkeiten: *land* ist abhängig von *interpret*

id	album	interpret	land
1	Repeater	Fugazi	USA
2	Red Medicine	Fugazi	USA

⇒ cd

id	album	interpret
1	Repeater	1

interpret

id	name	land
1	Fugazi	USA

BCNF: Nur abhängigkeiten vom Schlüssel
(Voll-)funktionale Abhängigkeit: B hängt von A ab, zu jedem Wert von A gibt es genau einen Wert von B ($A \rightarrow B$)
Teilweise funkt. Abh.: B hängt von A ab, aber auch von einem Teil eines zusammengesetzten Schlüssels.
Transitive Abhängigkeit: B hängt vom Attribut A ab, C hängt von B ab ($A \rightarrow B \wedge B \rightarrow C \Rightarrow A \rightarrow C$)
Denormalisierung: In geringere NF zurückführen (Verbessert Performance und reduziert Joins-Komplexität)
ANOMALIEN
Einfügeanomalie, Löschanomalie, Änderungsanomalie

DATA CONTROL LANGUAGE (DCL)

```
<role> ::= 'ALTER ROLE' <name> <priv> { ',' <priv> }  
<grant> ::= 'GRANT' <actions> 'ON' <object> 'TO' <grantees> [ 'WITH GRANT OPTION' ]  
<revoke> ::= 'REVOKE' [ 'GRANT OPTION FOR' ] <actions> 'ON' <object> 'FROM' <grantees> ( 'CASCADE' | 'RESTRICT' )  
<priv> ::= [ 'NO' ] ( 'CREATEDB' | 'CREATOROLE' | 'INHERIT' )  
<actions> ::= 'ALL' | 'SELECT' | 'DELETE' | 'TRIGGER' | ( ( 'INSERT' | 'UPDATE' | 'REFERENCES' ) ( ' ( <columns> ' ) ' ) <actions> ) { ',' <action> }  
<grantees> ::= <role_names> { ',' <role_name> }  
<object> ::= 'TABLE' | 'COLUMN' | 'VIEW' | 'SEQUENCE' | 'DATABASE' | 'FUNCTION' | 'SCHEMA'
```

Falls WITH GRANT OPTION: Der Berechtigte kann den Zugriff anderen Usern verteilen. → REVOKE ... CASCADE;

```
CREATE ROLE u WITH LOGIN PASSWORD ''; -- user  
GRANT INSERT ON TABLE t TO u WITH GRANT OPTION;  
ALTER ROLE u CREATOROLE, CREATEDB, INHERIT;  
CREATE ROLE r; -- group  
GRANT r TO u; -- put user u in group r  
REVOKE CREATE ON SCHEMA s FROM r;  
CREATE ROLE u PASSWORD '' IN ROLE r; -- equivalent
```

READ-ONLY USER

```
-- creating  
REVOKE CREATE ON SCHEMA public FROM PUBLIC;  
CREATE ROLE u WITH LOGIN ENCRYPTED PASSWORD '' NOINHERIT; -- don't inherit privileges  
GRANT SELECT ON ALL TABLES IN SCHEMA public TO u; -- read all new tables (also created by others):  
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT ON TABLES TO u; -- deleting  
REVOKE SELECT ON ALL TABLES IN SCHEMA public FROM u;  
ALTER DEFAULT PRIVILEGES IN SCHEMA public REVOKE SELECT ON TABLES FROM u;  
DROP USER u;
```

ROW-LEVEL SECURITY (RLS)

```
CREATE TABLE exams (  
id SERIAL, -- other fields...  
teacher VARCHAR(60) DEFAULT current_user  
);  
CREATE POLICY teachers_see_own_exams ON exams  
FOR ALL TO PUBLIC USING (teacher = current_user);  
ALTER TABLE exams ENABLE ROW LEVEL SECURITY;
```

DATA DEFINITION LANGUAGE (DDL)
Wichtig: NOT NULL wo notwendig nicht vergessen

```
CREATE SCHEMA s;  
CREATE TABLE t (  
id SERIAL PRIMARY KEY,  
name TEXT UNIQUE,  
grade DECIMAL(2,1) NOT NULL,  
fk INT FOREIGN KEY REFERENCES t2.id  
ON DELETE CASCADE,  
added TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
u VARCHAR(9) DEFAULT CURRENT_USER,  
CHECK (grade between 1 and 6)  
);  
ALTER TABLE t2 ADD CONSTRAINT c PRIMARY KEY (a, b);  
TRUNCATE/DROP TABLE t;
```

VERERBUNG

Tabelle pro Sub- und Superklasse:

```
CREATE TABLE sup ( -- 3.a  
id SERIAL PRIMARY KEY,  
name TEXT UNIQUE  
);  
CREATE TABLE sub1 (  
id SERIAL PRIMARY KEY,  
age INT  
);  
CREATE TABLE sub2 (  
id SERIAL PRIMARY KEY  
);  
ALTER TABLE sub1 ADD CONSTRAINT id FOREIGN KEY REFERENCES sup (id);  
ALTER TABLE sub2 ADD CONSTRAINT id FOREIGN KEY REFERENCES sup (id);
```

Tabelle pro Subklasse: Enthält jeweil. Subklassenattribute

```
CREATE TABLE sub1 ( -- 3.b  
id SERIAL PRIMARY KEY,  
name TEXT UNIQUE,  
age INT  
);  
CREATE TABLE sub2 (  
id SERIAL PRIMARY KEY,  
name TEXT UNIQUE  
);
```

Einzige Tabelle für Superklasse: Enthält alle Attribute

```
CREATE TABLE sup ( -- 3.c  
id SERIAL PRIMARY KEY,  
name TEXT UNIQUE,  
age INT  
);
```

JUNCTION TABELLEN

```
CREATE TABLE a_b(  
a INTEGER REFERENCES a(id),  
b INTEGER REFERENCES b(id),  
PRIMARY KEY(a, b)  
);
```

VIEWS
Resultate werden jedes mal dynamisch queried

```
CREATE VIEW v (id, u) AS SELECT id, u FROM t;  
-- complex query  
CREATE VIEW cheap_restaurant_view AS  
WITH big_restaurant AS (  
SELECT * FROM restaurant  
WHERE anzahl_plaetze >= 20  
)  
SELECT r.name AS restaurant_name, s.name,  
MIN(g.preis) AS cheap_gericht  
FROM big_restaurant r  
LEFT JOIN skigebiet s ON (s.id = r.skigebiet_id)  
LEFT JOIN menukarte m ON (r.id = m.restaurant_id)  
LEFT JOIN menu_gericht mg ON (m.id = mg.menu_id)  
LEFT JOIN gericht g ON (g.id = mg.gericht_id)  
WHERE ist_tagesmenu = true  
GROUP BY r.id, s.id, restaurant_name  
HAVING MIN(g.preis) >= 3  
ORDER BY cheap_gericht;
```

UPDATABLE VIEW
Views sind updatable wenn diese Kriterien erfüllt sind:
– Eine einzige «base tabelle»
– Keine aggregate, DISTINCT, GROUP BY, oder HAVING
– Alle Spalten müssen zur originalen Tabelle direkt gemappt werden können

MATERIALIZED VIEW
Speichert resultat auf Disk

```
CREATE MATERIALIZED VIEW mv AS SELECT * FROM t;  
REFRESH MATERIALIZED VIEW mv; -- refresh results
```

TEMPORÄRE TABELLEN

```
CREATE TEMPORARY TABLE temp_products (  
id SERIAL PRIMARY KEY,  
product_name TEXT  
);
```

```
INSERT INTO temp_products (product_name) VALUES  
( 'Product A' ), ( 'Product B' ), ( 'Product C' );
```

```
SELECT ts.product_name, ts.quantity FROM  
temp_sales ts JOIN temp_products tp ON  
ts.product_name = tp.product_name;
```

DATENTYPEN

```
CREATE TYPE grade AS ENUM('A','B','C','D','E','F');  
NUMERIC(4, 2) /* 99.99 */ NUMERIC(2, 1) /* 9.9 */  
VARCHAR(5) /* 'abcde' */ CHAR(5) /* 'abcde' */
```

DREIWEITIGE LOGIK (CURSED)

```
SELECT NULL IS NULL; -- true  
SELECT NULL = NULL; -- [unknown]
```

Typ	Beschreibung
INTEGER/INT	Integer (4 bytes)
BIGINT	Large integer (8 bytes)
SMALLINT	Small integer (2 bytes)
REAL	Single precision float (4 bytes)
NUMERIC(precision, scale)	Exact numeric of selectable precision
DECIMAL(precision, scale)	Alias for DECIMAL(precision, scale)
DOUBLE PRECISION	Double precision float (8 bytes)
SERIAL	Auto-incrementing integer (4 bytes)
BIGSERIAL	Auto-incrementing large integer (8 bytes)
SMALLSERIAL	Auto-incrementing small integer (2 bytes)
CHARACTER/CHAR(size)	Fixed-length, blank-padded string
VARCHAR(size)	Variable-length, non-blank-padded string
TEXT	Variable-length character string
BOOLEAN	Logical Boolean (true/false)
DATE	Calendar date (year, month, day)
TIME	Time of day (no time zone)
TIMESTAMP	Date and time (no time zone)
TIMESTAMP WITH TIME ZONE	Date and time with time zone
INTERVAL	Time interval
JSON	JSON data
UUID	Universally unique identifier
ARRAY OF base_type	Array of values

CASTING
Explizit

```
CAST(5 AS float8) = 5::float8  
SELECT 'ABCDEF6'::NUMERIC; -- error  
SELECT SAFE_CAST('ABCDEF6' AS NUMERIC); -- NULL
```

Implizit

```
SELECT 5 + 3.2; -- 5 is cast to 5.0 (numeric)  
SELECT 'Number' || 42; -- 42 is cast to '42'  
SELECT true AND 1; -- 1 is treated as true  
SELECT CURRENT_TIMESTAMP + INTERVAL '1 day';  
-- CURRENT_TIMESTAMP to date ^  
SELECT '100'::text + 1; -- '100' is cast to 100
```

DATA MANIPULATION LANGUAGE (DML)

```
<selects> ::= [ 'WITH' [ 'RECURSIVE' ] <with_query> [ ',' ... ] ]  
'SELECT' [ 'ALL' | 'DISTINCT' ] [ 'ON' ( <expression> [ ',' ... ] ) ]  
[ ' { '*' | <expression> [ [ 'AS' ] <output_name> ] }  
[ ' ... ] ] ]  
[ 'FROM' <from_item> [ ' ... ] ]  
[ 'WHERE' <condition> ]  
[ 'GROUP BY' [ 'ALL' | 'DISTINCT' ] <grouping_elem> [ ' ... ] ]  
[ 'HAVING' <condition> ]  
[ 'WINDOW' <window_name> 'AS' ( <window_def> ) [ ' ... ] ]  
[ { 'UNION' | 'INTERSECT' | 'EXCEPT' } [ 'ALL' | 'DISTINCT' ]  
<selects> ]  
[ 'ORDER BY' <expression> [ 'ASC' | 'DESC' ] [ 'USING' <op> ]  
[ 'NULLS' ( 'FIRST' | 'LAST' ) ] [ ' ... ] ] ]  
[ 'LIMIT' { <count> | 'ALL' } ]  
[ 'OFFSET' <start> [ 'ROW' | 'ROWS' ] ] ]
```

```
<from_item> ::= <table> [ '*' ] [ [ 'AS' ] <alias> [ ( ( <col_alias> [ ' ... ] ) ] ] ]  
[ 'LATERAL' ] ( <select> ) [ [ 'AS' ] <alias> [ ( ( <col_alias> [ ' ... ] ) ] ] ]  
<with_query_name> [ [ 'AS' ] <alias> [ ( ( <col_alias> [ ' ... ] ) ] ] ]  
<from_item> <join_type> <from_item> [ 'ON' <join_condition> |  
'USING' ( <join_column> [ ' ... ] ) [ 'AS' <join_using_alias> ] ]  
<from_item> 'NATURAL' <join_type> <from_item>  
<from_item> 'CROSS JOIN' <from_item>
```

with_query ::= <name> [((<col_name> [' ...])]]
'AS' (<select> | <values> | <insert> | <update> | <delete> | <merge>)
['USING' <cycle_path_col_name>]

JOINS

users (*u*)

actions (*a*)

INFO: FK *uid* in den Query-Resultaten unten aus Platzgründen ausgelassen

<i>id</i>	<i>name</i>
1	Alice
2	Bob

<i>id</i>	<i>uid</i>	<i>action</i>
7	1	LOGIN
8	2	VIEW
9	4	LOGIN

